

Introduction to Artificial Intelligence

Week 6

Agenda

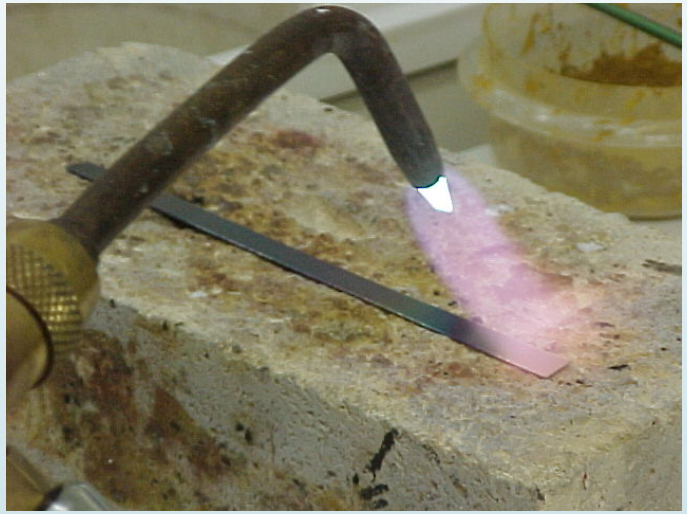
- Simulated Annealing
- Exploration and Exploitation
- A*
- Tabu Search

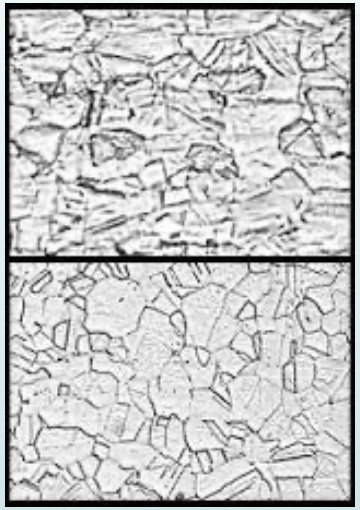
Learning by Searching

Part II

Simulated Annealing

- Inspired by metallurgy. Based on the properties of Metal working
- Metal is heated and cooled in order for it to be made workable
- Crystal lattice atoms are more active when temperature is higher
- Can be considered as a mixture of Hill Climber and Random Walk



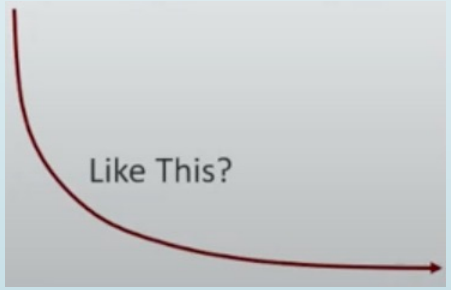


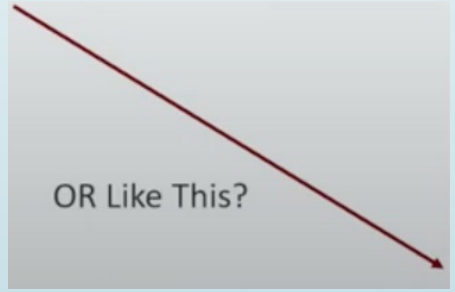
Simulation of This Process

- Set down a number of search points
- Points move dependent upon the temperature applied
- They move about the space based on a positive temperature which is slowly reducing over time
  - This allows for large movement at the beginning of the algorithm and reduces into smaller and smaller movements as time goes on
- Two Types
  - Temperature controls size of step
  - Temperature controls the acceptance of a step

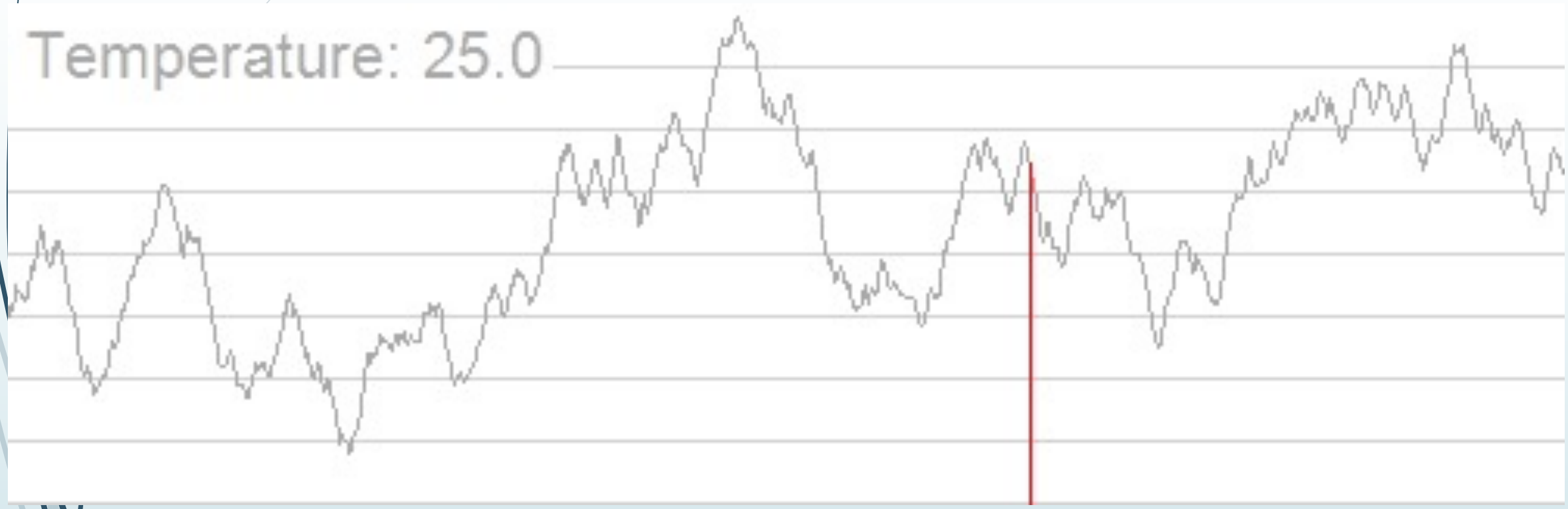
Simulated Annealing

- Start at an initial temperature value (usually something **hot**) and create a random initial solution
- Until temperature is **cool**, usually 0
  - Make a change to the current solution (neighbour) based on the temperature
    - Current location + Rand_Distribution(*T*), where *T* is temperature
    - If new state x_2 is better than x_1 ($f(x_1) < f(x_2)$), then move to x_2
    - Else, move to x_2 with probability $P(x_1 \rightarrow x_2) = e^{(f(x_1)-f(x_2))/T}$
  - Decrease the temperature *T*
    - This can be done according to different formulas, but has to be gradual process
    - Decreasing T reduces hops over the search space





Temperature Controls the Step Size



Exploring and Exploiting

- *Exploration* – the want for a search method to find new areas of the search space which have not been visited yet
- *Exploitation* – the want for a search method to locally search about the best known areas in the space in order to refine these current solutions
- All search algorithms will have some mix of both properties
- Note that if we only have a limited number of objective tests available, then we have a zero-sum game – we can explore or we can search with a test

Exploration or Exploitation?

- We create a simulated annealing which has a reheat if we are not seeing a 10% improvement in the solution

Exploration

- The reheat increases the step size
- The reheat increases the probability we accept worst solutions

Exploration or Exploitation?

- After the application of a search we look at the least significant digit and test from 0-9 taking the best final solution

Exploitation

- Looking at the least significant digit we are looking in a close locality to the previous, making the assumption that close parameter settings generate closer outputs

Parallel

- Hill Climbers and Simulated Annealing are trivially parallel
- We can place one of these searching units onto a different ‘core’ processing unit without any issues
- Hill Climbers and Simulated Annealing use the idea of a solution neighbourhood, so we can divide the space

A* Search

- Fast well-established algorithm for game AI
  - Pathways
  - Movement of NPC (non-player character)
  - RTS (real-time strategy) movement
    - Usually one member of a flock
- Works in areas where we do not know the connectivity
  - Makes it very suitable for PCG – because we do not need nodes defined by the generator – but can still understand the measurements
- Can take into account multiple different connection topologies and styles of movement
- Large number of iterative improvements and problem domain specific implementations with heuristics

A* Formula

- $f(n) = g(n) + h(n)$
  - $g(n)$ is the real path from the initial position to the current node
  - $h(n)$ is the estimated path from the current node to the goal position
    - To make Heuristics optimal, it has to be admissible and consistent

REMINDER

Heuristics Admissibility and Consistency

- In path-finding problems heuristics may be *admissible* or *inadmissible*
- Admissibility means that heuristics is not overestimating the solution
  - Needs to be lower than or equal to the actual cost of reaching the goal
- Not each heuristic is obliged to be admissible. However, inadmissible heuristic does not guarantee the optimal solution
- In path-finding problems a heuristic is considered to be *consistent* (or *monotone*) if the estimated cost from one node to the successor node, added to the estimated cost from the successor node to the goal is less than or equal to the estimated cost from the current node to the goal state
  - $E(f(x_i \rightarrow x_{i+1})) + E\big(f(x_{i+1} \rightarrow x_{goal})\big) \leq E\big(f(x_i \rightarrow x_{goal})\big)$ → consistent heuristics
- All consistent heuristics are admissible heuristics, however, all admissible heuristics are not necessarily consistent heuristics

A* Algorithm

A node contains a location, a cost, and a parent

Initialize sets CLOSED and BLOCKED to be null

Initialize a priority queue based on cost of a node called OPEN to be the starting location, set cost = 0

While OPEN is non-empty do

Current <- Pop OPEN.top

For all neighbouring locations X

If not in (BLOCKED or CLOSED)

If not in OPEN

X.cost = current.cost + costFromCurrentToX
X.parent = current

Push X to OPEN

Else if X is in OPEN

If current.cost + costFromCurrentToX < OPEN.X.cost
// we found a better path

OPEN.X.cost = current.cost + costFromCurrentToX

OPEN.X.parent = current

END FOR ALL X

Push Current to CLOSED

A* for PATH FINDING

- REPORT_PATH (START, FINISH)
  - STACK PATH initialized to NULL
  - RUN A* (START)
  - Find FINISH in CLOSED
  - Current <- FINISH
  - While Current not START
    - PUSH Current to PATH
    - Current <- Current.parent
  - PRINT START
  - While STACK not NULL
    - PRINT POP PATH

Rover World

Rover



Obstacle

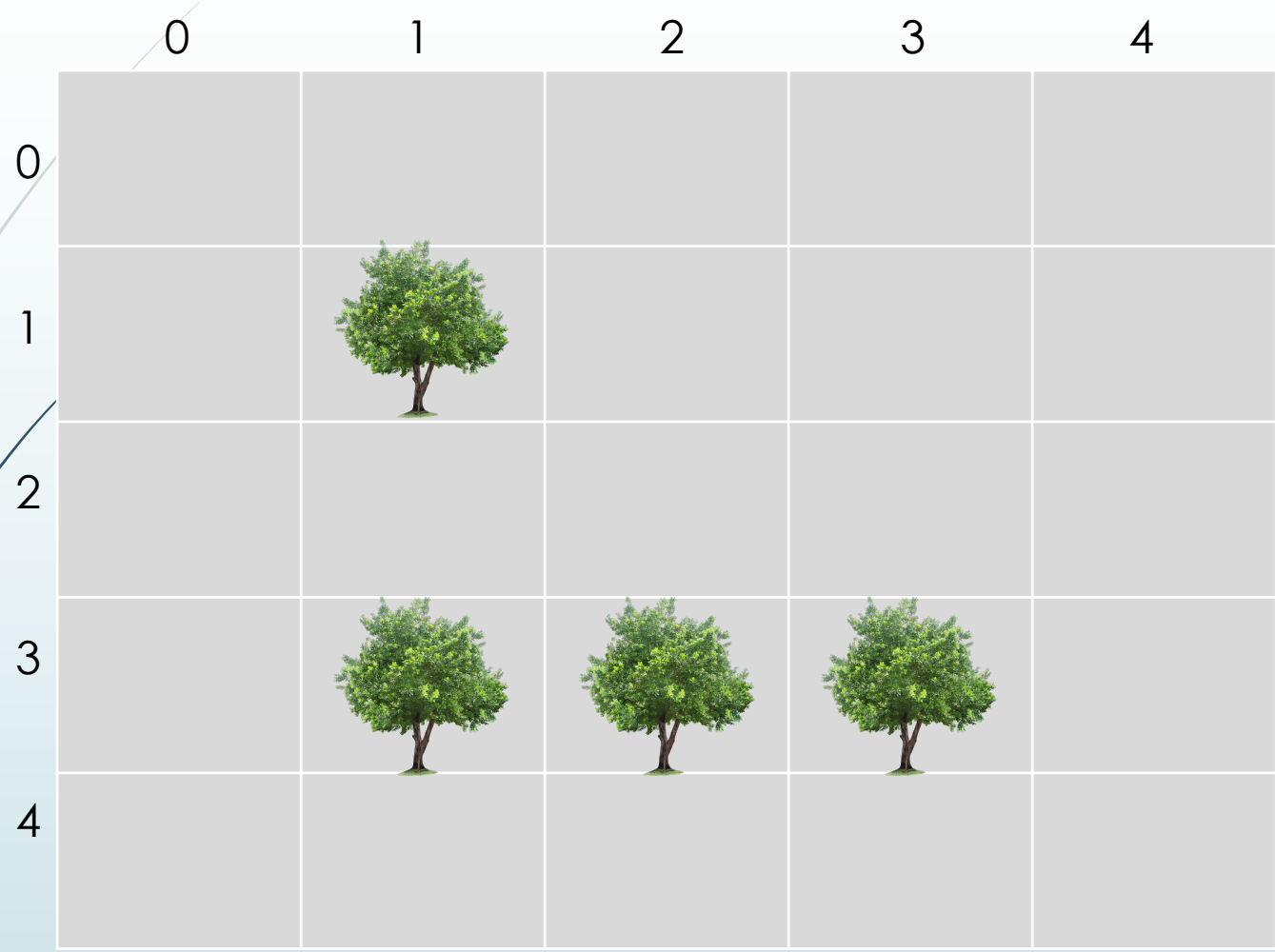


Goal

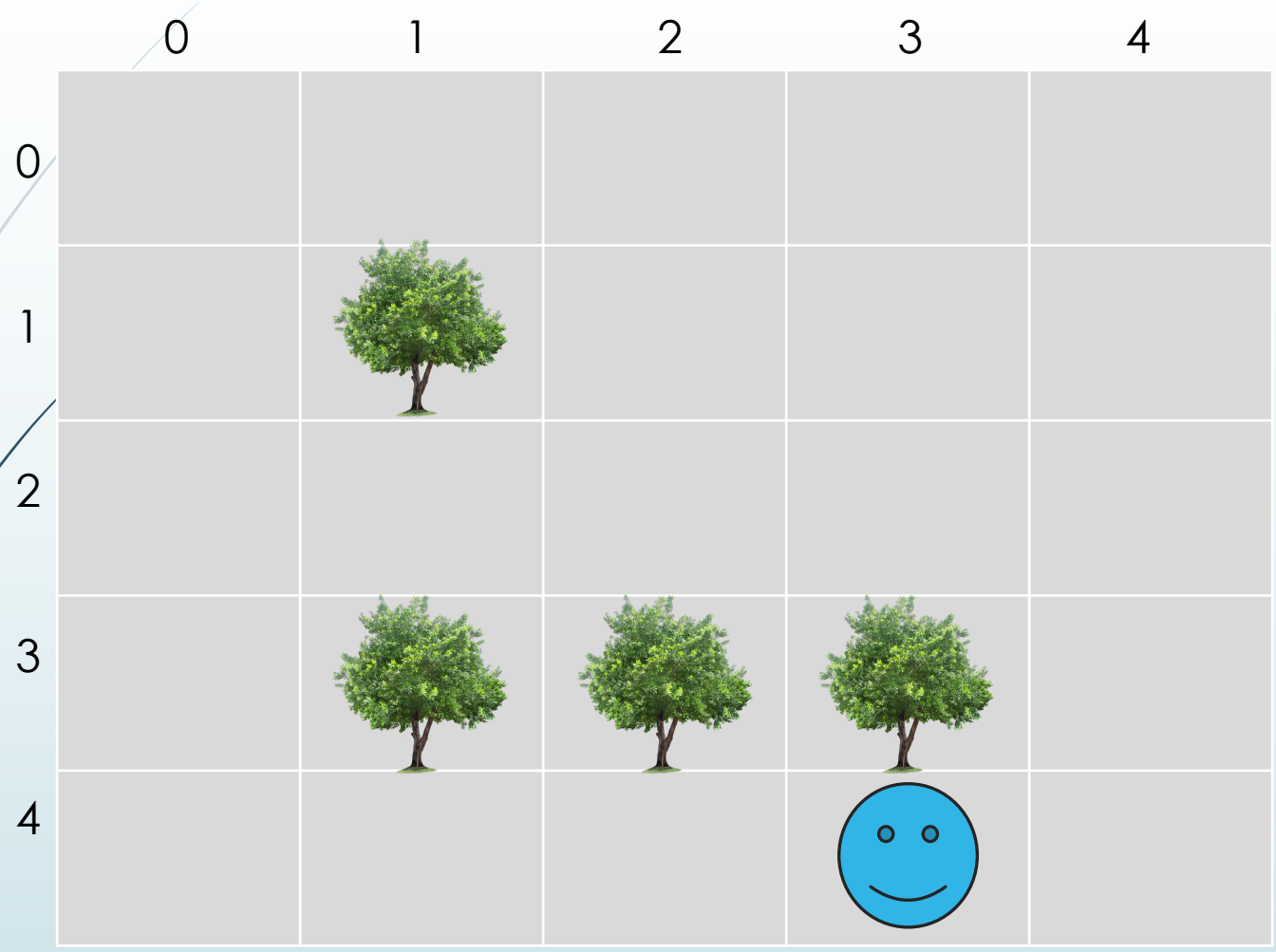


	0	1	2	3	4
0					
1					
2					
3					
4					

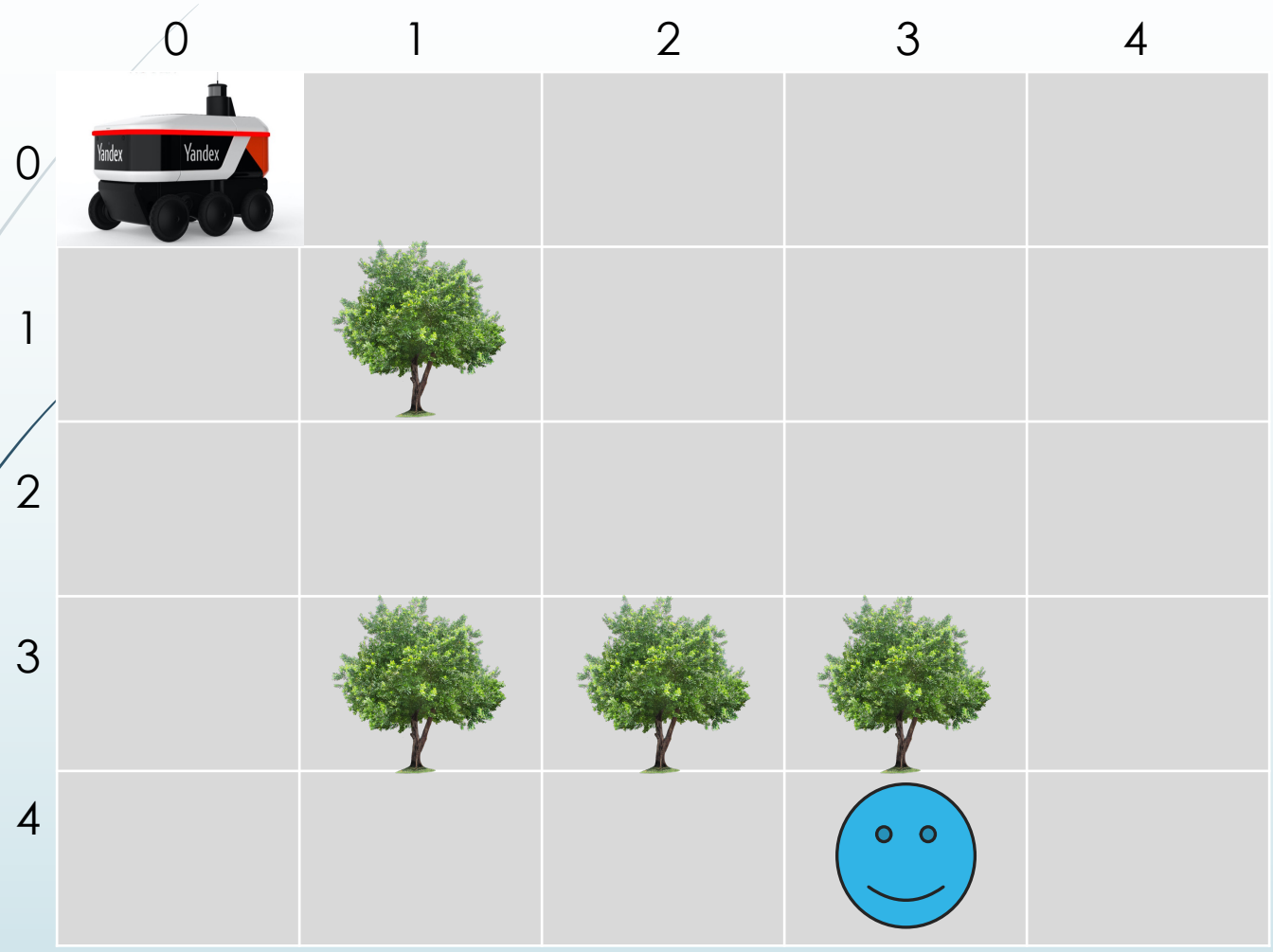
World with Obstacles



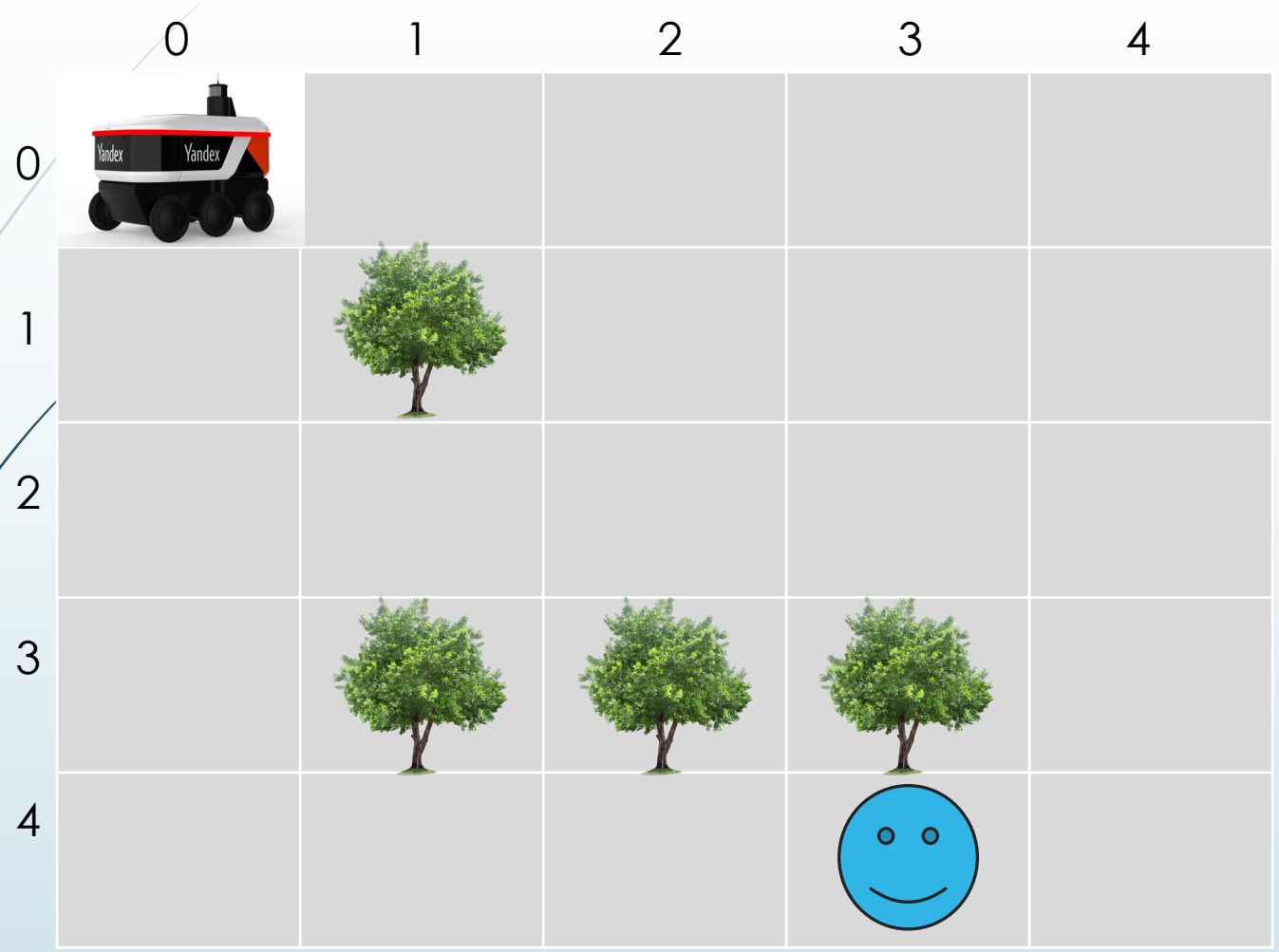
World with Obstacles and Goal



World with Obstacles, Goal and Starting Position



World with Obstacles, Goal and Starting Position



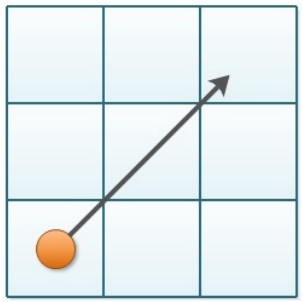
- To find the path to the customer Rover requires A* algorithm to be applied
- Assume that the Rover can move horizontally (10p), vertically (10p) and diagonally (14p)
- For heuristics we use a **Euclidean distance without mantissa** from the current cell till the goal
- For each cell x costs g(x), h(x) and f(x) = g(x) + h(x) have to be assigned

g(x) h(x)
f(x)

Types of Heuristics

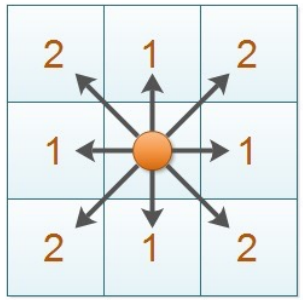
- Correct choice of heuristics defines the optimality of A*
- If diagonal moves are allowed and the cost is the same as to vertical and horizontal moves than Chebyshev Distance is preferable, it is also called a Straight Line Distance
- If only 4 directions, than Manhattan Distance is more logical

Euclidean Distance



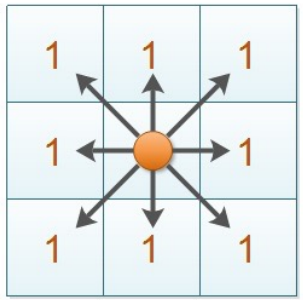
$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Manhattan Distance



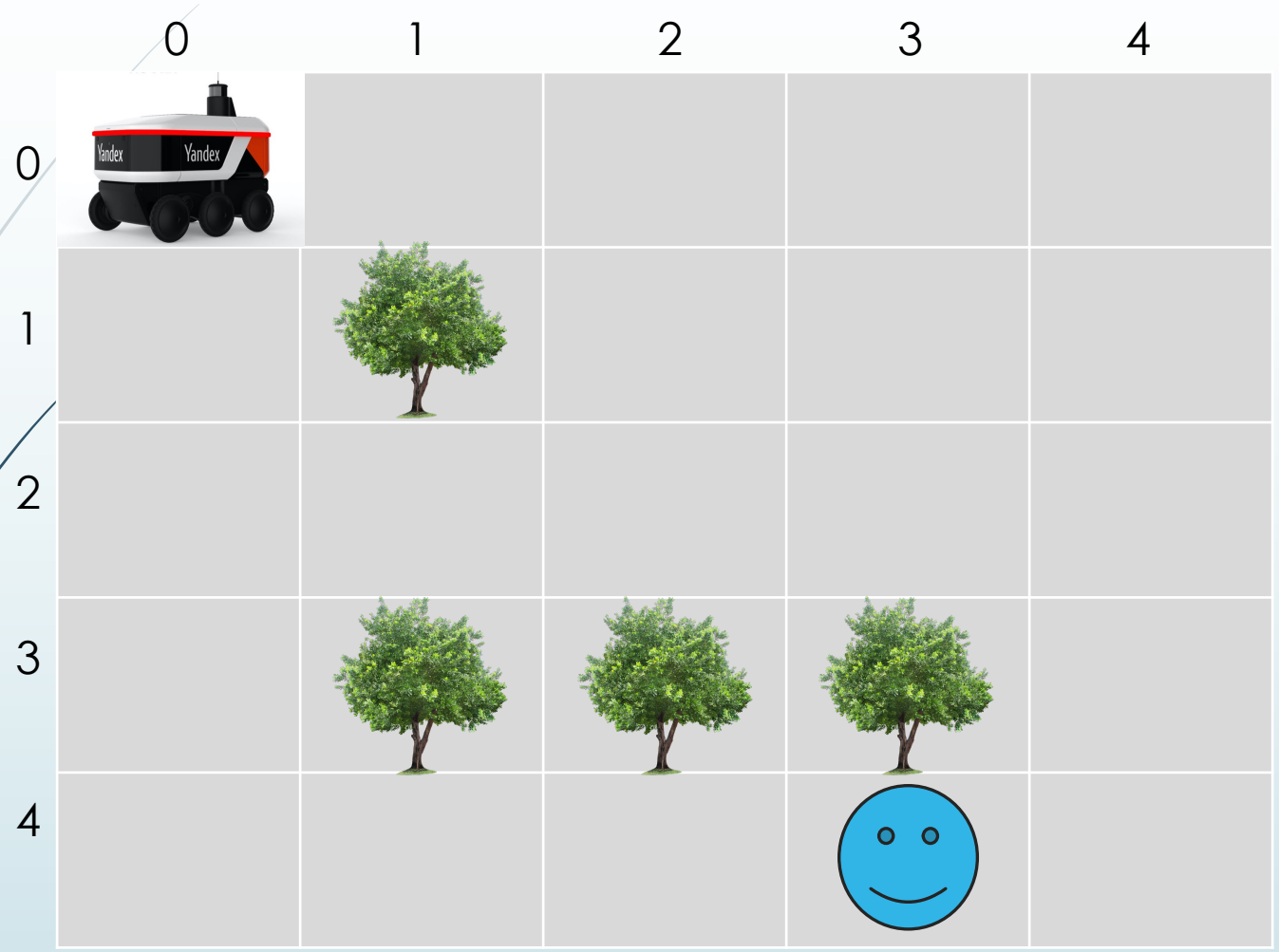
$$|x_1 - x_2| + |y_1 - y_2|$$

Chebyshev Distance



$$\max(|x_1 - x_2|, |y_1 - y_2|)$$

Add Initial Point to Open List and Assign Cost

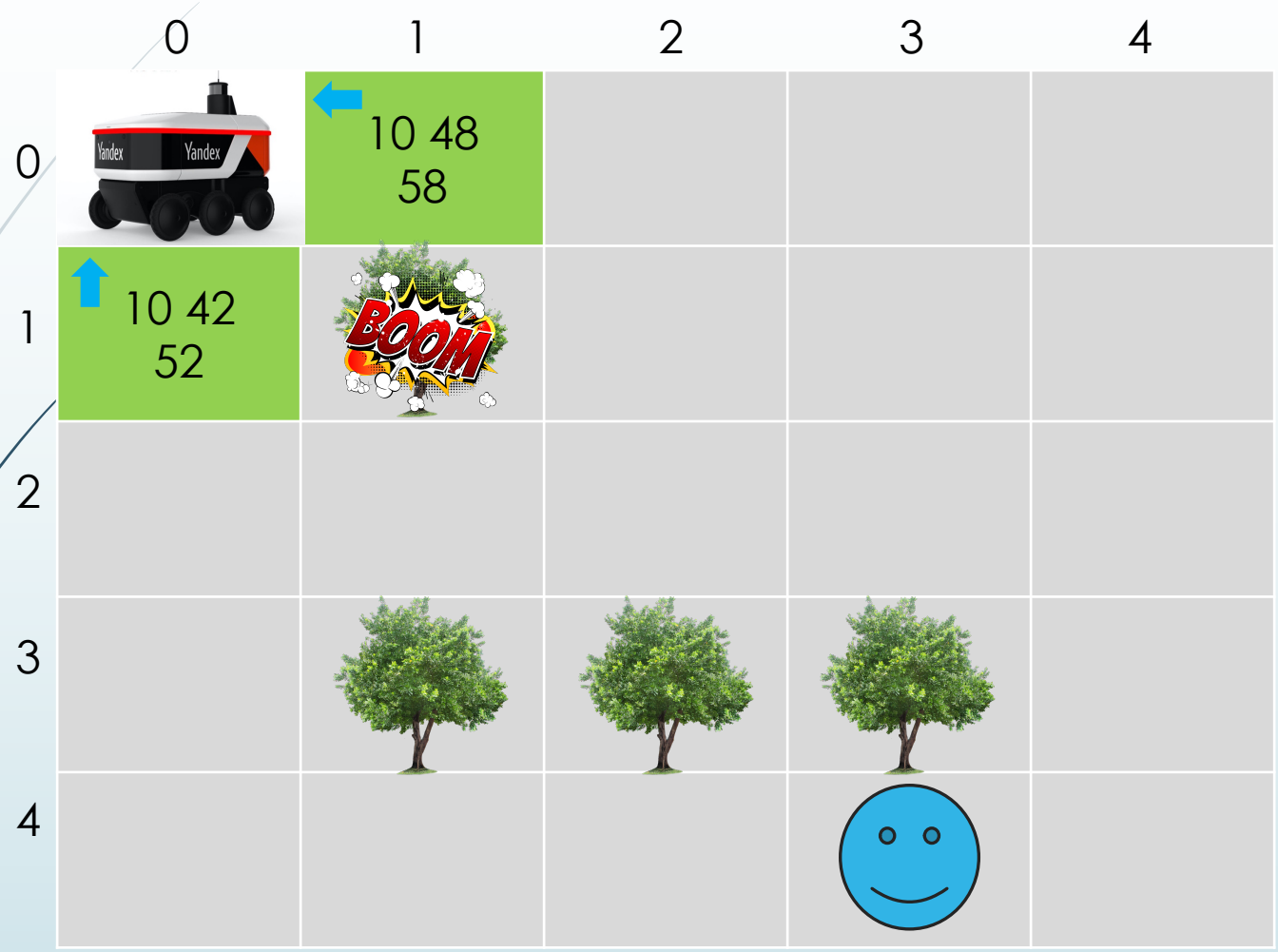


CLOSED:
NULL

OPEN:
(0,0) – 0

g(x) h(x)
f(x)

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

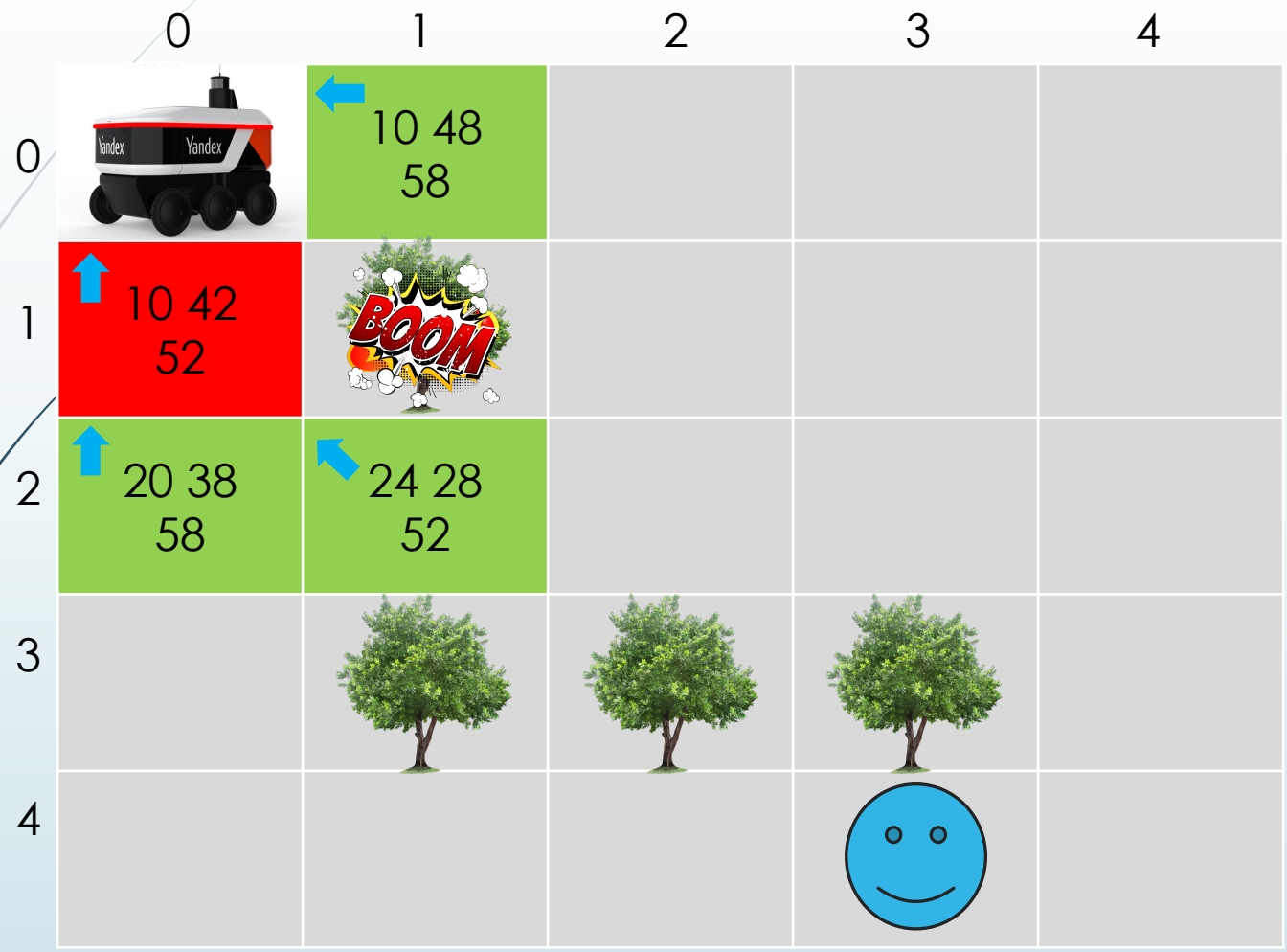


CLOSED:
(0,0) – 0 – NEW

OPEN:
(1,0) – 52 – NEW
(0,1) – 58 – NEW
(1,1) – COLLISION

For simplicity we can assign infinitely big value to each obstacle cell

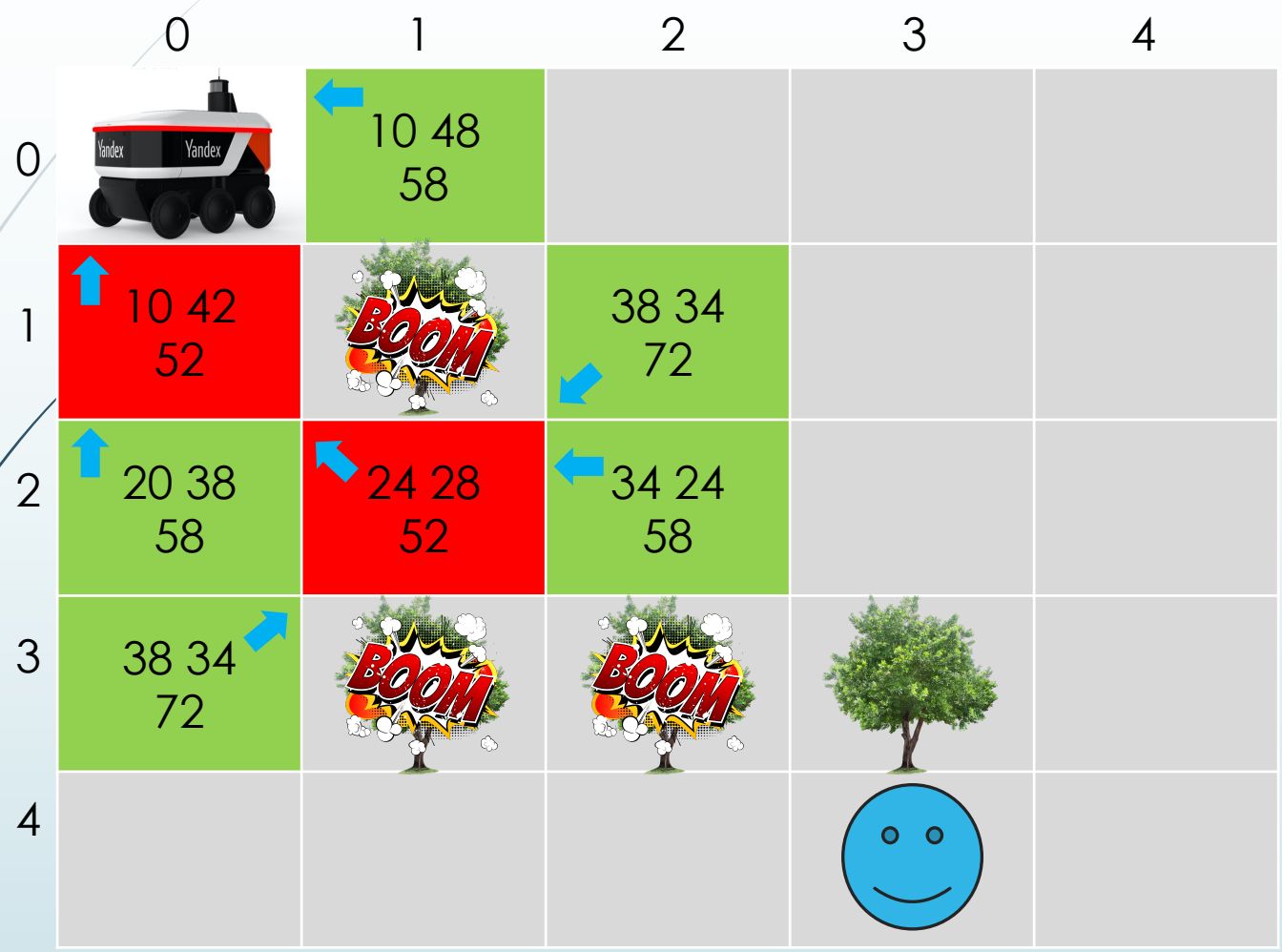
Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52 – NEW

OPEN:
(0,1) – 58
(1,1) – ∞
(2,0) – 58 – NEW
(2,1) – 52 – NEW

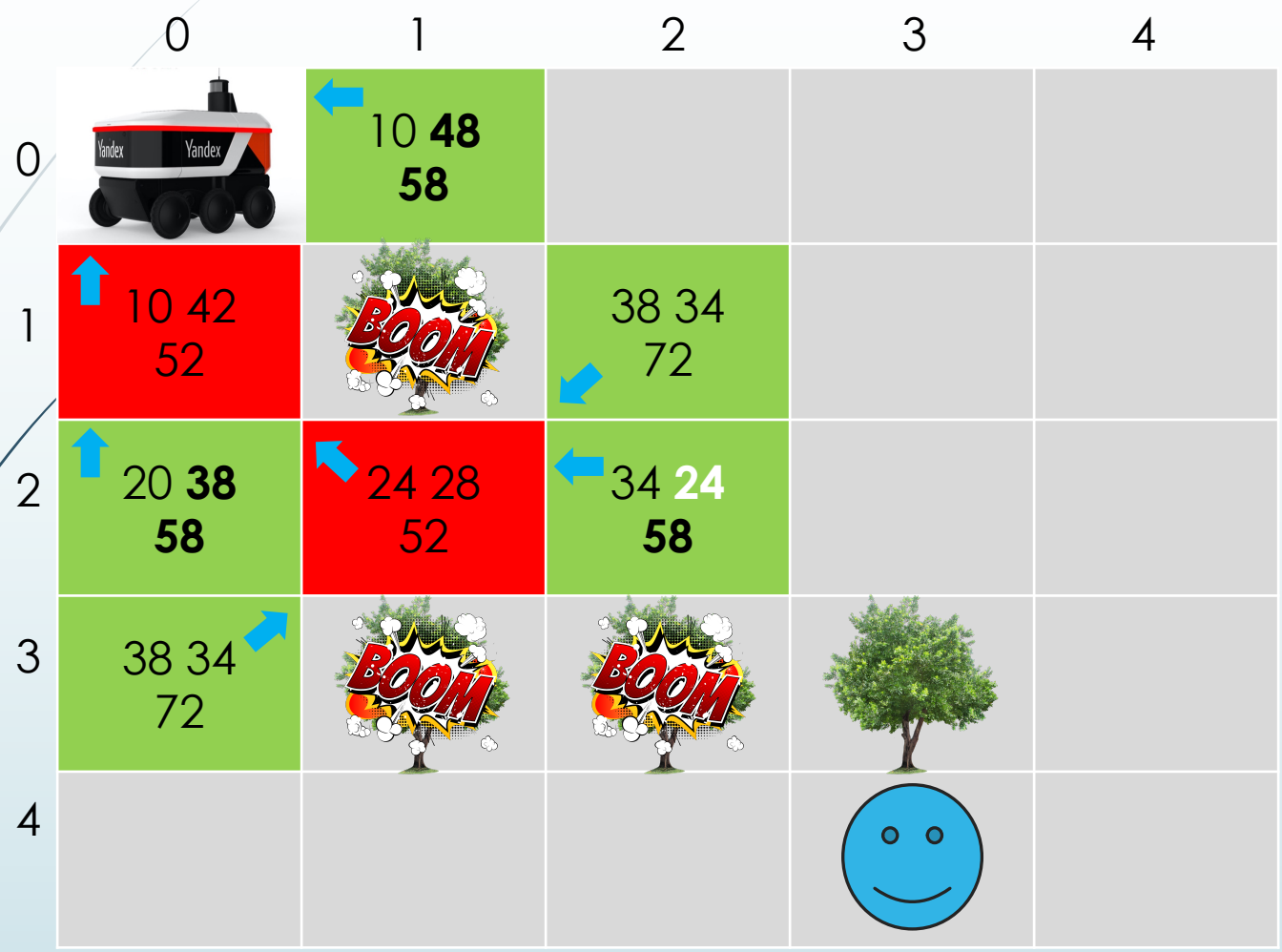
Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52 – NEW

OPEN:
(0,1) – 58
(1,1) – ∞
(2,0) – 58
(3,1) – ∞ – NEW
(3,2) – ∞ – NEW
(3,0) – 72 – NEW
(2,2) – 58 – NEW
(1,2) – 72 – NEW

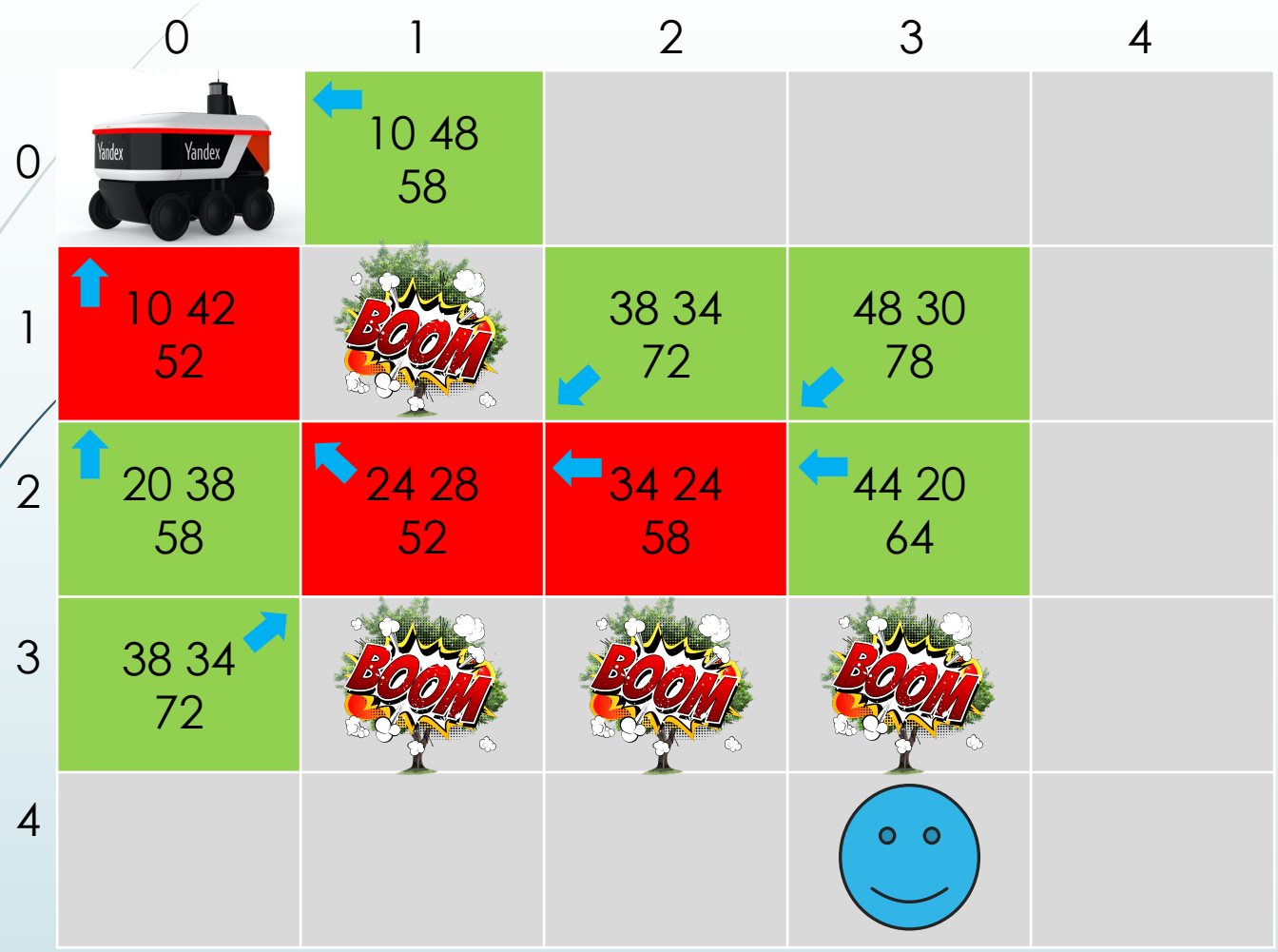
Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52 – NEW

OPEN:
(0,1) – 58
(1,1) – ∞
(2,0) – 58
(3,1) – ∞ – NEW
(3,2) – ∞ – NEW
(3,0) – 72 – NEW
(2,2) – 58 – NEW
(1,2) – 72 – NEW

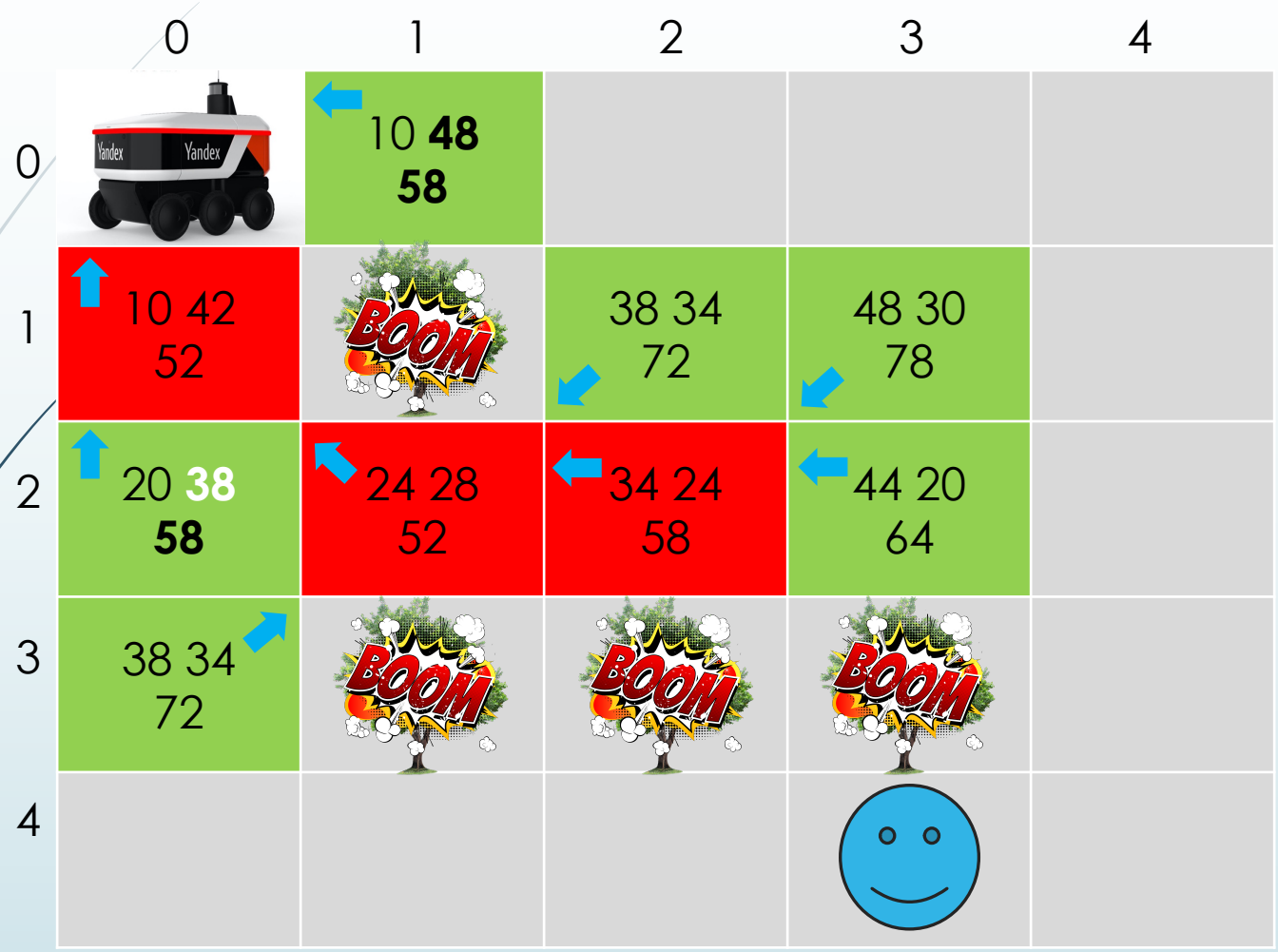
Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58 – NEW

OPEN:
(0,1) – 58
(1,1) – ∞
(2,0) – 58
(3,1) – ∞
(3,2) – ∞
(3,0) – 72
(1,2) – 72
(2,3) – 64 – NEW
(1,3) – 78 – NEW

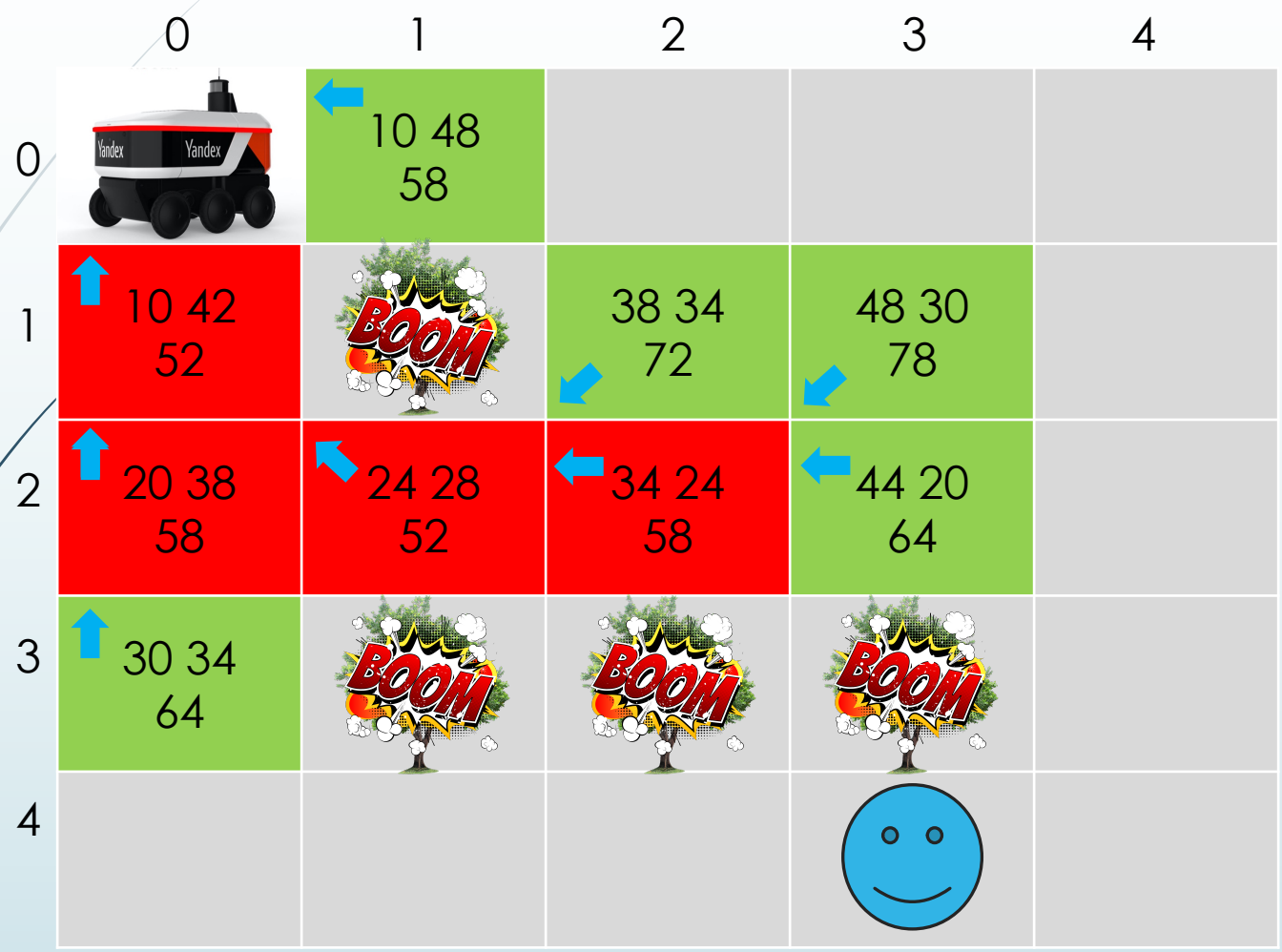
Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58 – NEW

OPEN:
(0,1) – 58
(1,1) – ∞
(2,0) – 58
(3,1) – ∞
(3,2) – ∞
(3,0) – 72
(1,2) – 72
(2,3) – 64 – NEW
(1,3) – 78 – NEW

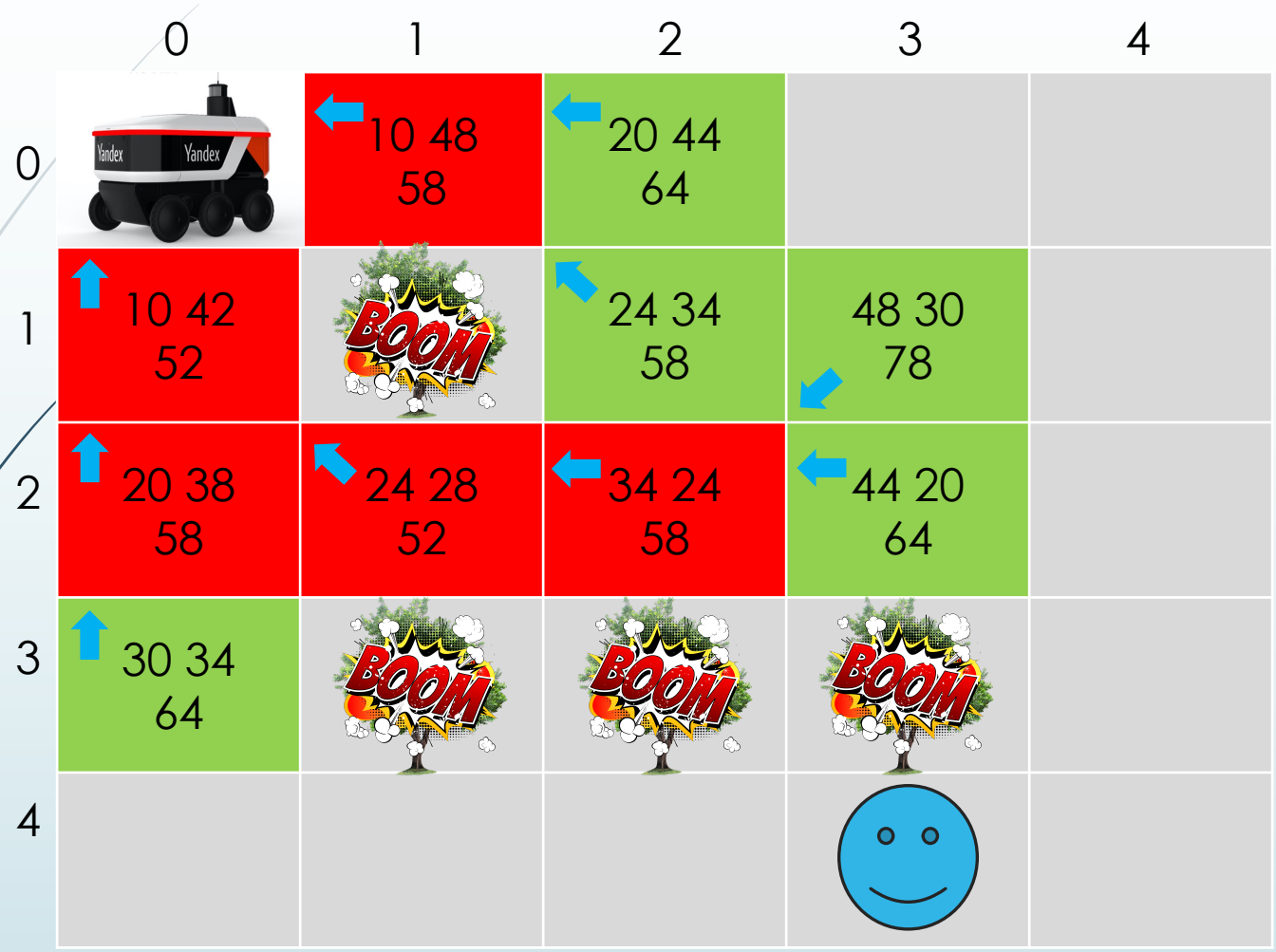
Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58 - NEW

OPEN:
(0,1) – 58
(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(3,0) – 64 – UPD
(1,2) – 72
(2,3) – 64
(1,3) – 78

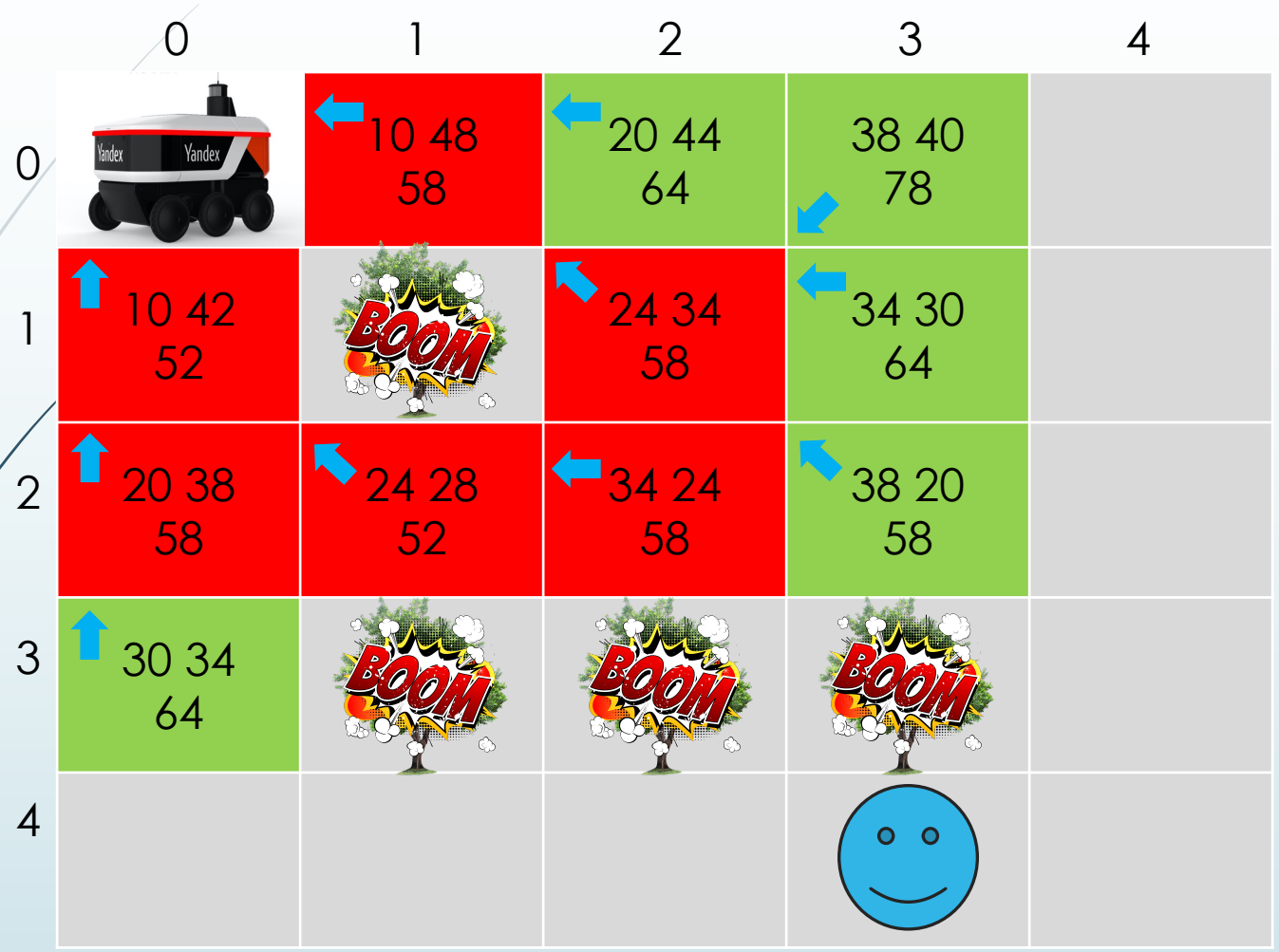
Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58 – NEW

OPEN:
(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(3,0) – 64
(1,2) – 58 – UPD
(2,3) – 64
(1,3) – 78
(0,2) – 64 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours



CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58 – NEW

OPEN:
(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(3,0) – 64
(2,3) – 58 – UPD
(1,3) – 64 – UPD
(0,2) – 64
(0,3) – 78 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 44 64	38 40 78	
1	10 42 52		24 34 58	34 30 64	52 34 86
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4					

CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58 – NEW

OPEN:
(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(3,0) – 64
(1,3) – 64
(0,2) – 64
(0,3) – 78
(1,4) – 86 – NEW
(2,4) – 72 – NEW
(3,4) – 66 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 **44** **64**	38 40 78	
1	10 42 52		24 34 58	34 **30** **64**	52 34 86
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 **34** **64**				52 14 66
4					

CLOSED:
(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58 – NEW

OPEN:
(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(3,0) – 64
(1,3) – 64
(0,2) – 64
(0,3) – 78
(1,4) – 86 – NEW
(2,4) – 72 – NEW
(3,4) – 66 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 44 64	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4					

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(3,0) – 64
(0,2) – 64
(0,3) – 78
(1,4) – 78 – UPD
(2,4) – 72
(3,4) – 66
(0,4) – 92 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 **44** **64**	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 **34** **64**				52 14 66
4					

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(3,0) – 64
(0,2) – 64
(0,3) – 78
(1,4) – 78 – UPD
(2,4) – 72
(3,4) – 66
(0,4) – 92 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 44 64	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 20 64			

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70 – NEW
(4,1) – 64 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 **44** **64**	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 **20** **64**			

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70 – NEW
(4,1) – 64 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 44 64	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 20 64	54 10 64		

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64
(4,1) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70
(4,2) – 64 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 **44** **64**	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 20 64	54 **10** **64**		

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64
(4,1) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70
(4,2) – 64 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 44 64	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 20 64	54 10 64	64 0 64 	

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64
(4,1) – 64
(4,2) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70
(4,3) – 64 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 **44** **64**	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 20 64	54 10 64	64 **0** **64** 	

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64
(4,1) – 64
(4,2) – 64 – NEW

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70
(4,3) – 64 – NEW

Move Lowest Cost to Closed with Smaller Heuristics and Search Neighbours

	0	1	2	3	4
0		10 48 58	20 44 64	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 20 64	54 10 64	64 0 64 	

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64
(4,1) – 64
(4,2) – 64
(4,3) – 64 – AIM

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70

Goal is Found

	0	1	2	3	4
0		10 48 58	20 44 64	38 40 78	48 44 92
1	10 42 52		24 34 58	34 30 64	44 34 78
2	20 38 58	24 28 52	34 24 58	38 20 58	48 24 72
3	30 34 64				52 14 66
4	40 30 70	44 20 64	54 10 64	64 0 64 	

CLOSED:

(0,0) – 0
(1,0) – 52
(2,1) – 52
(2,2) – 58
(2,0) – 58
(0,1) – 58
(1,2) – 58
(2,3) – 58
(1,3) – 64
(3,0) – 64
(4,1) – 64
(4,2) – 64
(4,3) – 64 – AIM

OPEN:

(1,1) – ∞
(3,1) – ∞
(3,2) – ∞
(0,2) – 64
(0,3) – 78
(1,4) – 78
(2,4) – 72
(3,4) – 66
(0,4) – 92
(4,0) – 70

Improvements of A*

- Dealing with other units
  - Treat stopped units as unpassable
  - Treat moving units nearby as unpassable
  - More complexity – prediction of their movement for avoidance – query their path
- Cost of movement
  - Taken into account in the cost function
- Non-grid setting
- Open World
  - Enforce a grid or graph
  - Develop pathways explicitly for longer movements

Tabu Search

- Created by Fred Glover in 1986
- Idea coming from a social tabu – never return to where I once was
  - Heuristic – where I have been is not where I want to go
- Search is conducted by single point changes in the candidate solution with a list of previous places that the search was at, a Tabu List
- The tabus are feature sets of the parameters
- Searches locally, then places the locality on the tabu areas of the search – forcing it to move away

Tabu Tenure and Aspiration Criterion

- *Tabu tenure* is the number of iterations, during which tabu is considered
  - A memory for the last actions not be reverted back for some iterations
  - Chosen by programmer
- *Aspiration criterion* allows for potential solutions with tabu and better than best found to skip the tabu
  - This sets tabu tenure to max value, does not matter how small or big it was before
  - Can consider only significant difference to break the tabu
- *Stopping criterion* for algorithm is usually the attempt limit or a score threshold

Tabu Advantages and Disadvantages

- Advantages:
  - Accepts non-improving solutions to escape from local optimum
  - Can be applied to both discrete and continuous solution spaces
- Disadvantages:
  - Too many parameters to be determined
  - Number of iterations may be very large
  - Global optimum is not guaranteed to be found

Tabu Loop

- Set a current position, set best to this position, set a tabu list to null
- While not at a stopping condition
  - Create a candidate list from neighbours
  - Check if the candidates are on the tabu list. If they are, remove them, if aspiration criterion is not satisfied
  - Find the best candidate
  - Move to this as current position
  - Test the best candidate against the best
    - If it is the new best – replace
    - Create a tabu by looking at the differences in the features between this new best and the previous best
    - Produce a tabu restriction based on this difference
    - Add this tabu to the tabu list
    - Check tabu list length and remove old tabus based on tabu tenure

Example - Tabu

- String of 5 parameters – containing numbers 0-9
- Tabu List is *,*,3,*,*
- New Tabus are created by adding one point where the best strings matched
- Best is *0,3,3,2,1* scoring 14 (the way of scoring is omitted for simplicity)
- Local neighbourhood is +/-1 from each parameter
- Generate the new list and get members' scores. For example, top 3:
  - *0,2,3,2,0* - **17**
  - *0,3,4,2,1* - **18**
  - *0,4,4,2,0* – **20** (best in the new list)

Compare and Add Tabu

- ***0***,*3*,*3*,***2***,*1* – previous best (score 14)
- ***0***,*4*,*4*,***2***,*0* – new best (score 20)

Compare and Add Tabu

- ***0***,*3*,*3*,***2***,*1* – previous best (score 14)
- ***0***,*4*,*4*,***2***,*0* – new best (score 20)
- *0*,*,*,*2*,*
- Add the tabu of *0*,*,*,*,* or *,*,*,*2*,*
- New Tabu list of *,*,*3*,*,* and (*0*,*,*,*,* or *,*,*,*2*,*)

Issues with Too Many Tabus

- Small sets of tabus might make it impossible to move
- In our example, with 3 tabus all happening about the same value, it would lock movement completely (lock upper and lower move, then move between and make that tabu)
- Rationalize the Tabus – Costly
- Only allow a small enough number of Tabus – where is this necessary value? Is it too low? How often will it happen?
- Aspiration criteria – Accept Tabu moves if they SIGNIFICANTLY increase fitness

Summary

- Simulated Annealing
- Exploration and Exploitation
- A*
- Tabu Search

Next week: MCTS

References

- A* pathfinding
- Tabu search